

MOST COMMONLY ASKED INTERVIEW QUESTIONS

PROGRAMMING

1. Write a function to check if a number is a palindrome.
2. Find the largest of three numbers using a function.
3. Create a function to compute the factorial of a number recursively.
4. Implement a program to remove duplicates from a list.
5. Write a function to return the second largest number in a list.
6. Reverse a string without using built-in functions.
7. Count the number of vowels in a string
8. Write a program to check if a string is an anagram of another.
9. Implement a basic calculator using functions.
10. Write a function to merge two dictionaries.
11. Count the frequency of each word in a sentence.
12. Convert a binary string to a decimal number.
13. Swap two numbers without using a temporary variable.
14. Write a function to check if a number is a prime.
15. Sort a list of tuples by the second element.
16. Write a program to reverse the words in a sentence.
17. Generate Fibonacci numbers up to n terms.
18. Write a lambda to square every number in a list.
19. Use `map()` to convert a list of strings to integers.
20. Filter even numbers from a list using `filter()`.
21. Reduce a list to its sum using `reduce()`.
22. Find the common elements in two lists.
23. Write a program to flatten a nested list.

24. Convert a list of integers into a single integer.
25. Implement a function to check Armstrong number.
26. Write a program to convert Celsius to Fahrenheit.
27. Print the multiplication table of a number using a function.
28. Remove punctuation from a given string.
29. Count the number of digits in an integer.
30. Write a function to calculate LCM of two numbers.
31. Write a program to convert a string to title case manually.
32. Create a list of squares using list comprehension.
33. Find all prime numbers between 1 and 100.
34. Write a function that returns a dictionary of squares from 1 to n.
35. Check whether a string is a valid identifier.
36. Use bitwise AND, OR, and XOR on two numbers.
37. Implement a decorator that prints the execution time of a function.
38. Write a function to extract only numbers from a mixed string.
39. Write a function that accepts variable length arguments and returns their sum.
40. Implement a recursive function to compute the GCD of two numbers.
41. Check if a string contains all unique characters.
42. Use `zip()` to pair elements from two lists.
43. Write a function that returns the intersection of two sets.
44. Create a program to simulate a simple login system with 3 tries.
45. Find the most frequent character in a string.
46. Write a function to convert a number to binary using recursion.
47. Create a nested dictionary from two lists.
48. Write a program to rotate a list left by k elements.
49. Check if a list is sorted in ascending order.
50. Create a generator for even numbers up to n.
51. Write a program to find the sum of the diagonal elements in a matrix.
52. Implement matrix transpose using nested list comprehension.
53. Remove all whitespace from a string.
54. Write a function to chunk a list into sublists of size n.
55. Write a function that accepts a string and counts the number of uppercase and lowercase letters.

56. Find the longest word in a sentence.
57. Implement a stack using list methods.
58. Implement a queue using collections.deque.
59. Write a program to simulate rolling a dice.
60. Count the number of lines in a file.
61. Write a function to read and reverse lines from a text file.
62. Sort a list using bubble sort.
63. Implement binary search for a sorted list.
64. Write a function to replace all occurrences of a substring in a string.
65. Use dictionary comprehension to invert keys and values.
66. Extract domain name from an email address.
67. Write a function to split a string into chunks of length n.
68. Implement a Caesar cipher encryption and decryption.
69. Use `enumerate()` to iterate with index over a list.
70. Write a function to count the number of times a function is called.
71. Find the common prefix of a list of strings.
72. Simulate a basic calculator that takes expression as input.
73. Remove all non-digit characters from a string.
74. Create a list of all substrings of a string.
75. Write a program to find the second most frequent word in a file.
76. Implement a custom exception and raise it under a condition.
77. Write a program to validate an IP address.
78. Find the smallest missing positive integer from a list.
79. Use a set to remove duplicates and sort the result.
80. Write a function that checks for balanced parentheses.
81. Group words that are anagrams of each other.
82. Write a function to convert Roman numerals to integers.
83. Check whether a list is a palindrome.
84. Create a histogram from a dictionary.
85. Implement insertion sort on a list.
86. Reverse a dictionary (values become keys).
87. Merge k sorted lists into a single sorted list.

88. Write a function to flatten a dictionary.
89. Generate random password of length `n` using `random` module.
90. Calculate factorial using `lambda` and `reduce`.
91. Simulate tossing a coin 100 times and count heads/tails.
92. Write a program to find the longest increasing subsequence.
93. Rotate a matrix by 90 degrees clockwise.
94. Find the sum of even and odd digits in a number.
95. Convert a number to its English word representation.
96. Write a function to find the power of a number without `**`.
97. Implement a function to find the `nth` prime number.
98. Write a function that mimics the behavior of `range()`.
99. Implement quicksort in .
100. Calculate the average of numbers from user input.
101. Replace spaces in a string with hyphens.
102. Write a program that finds the square root of a number without using `math.sqrt`.
103. Implement a digital clock that shows time every second.
104. Write a program to find all palindromic substrings of a string.
105. Find the number of trailing zeroes in factorial of a number.
106. Convert a list of strings to a CSV string.
107. Implement a function to compress a string using counts of repeated characters.
108. Write a function to evaluate a postfix expression.
109. Extract all hashtags from a string.
110. Count frequency of characters using `collections.Counter`.
111. Reverse a string using a stack.
112. Implement a cache using dictionary with a size limit.
113. Write a function to multiply all elements in a nested list.
114. Count how many times a function is called using a decorator.
115. Find duplicate files in a directory by content.
116. Write a program that mimics the UNIX `wc` command.
117. Create a function that finds the median of a list.
118. Simulate a progress bar using `print()` and time delay.
119. Detect if a string is a valid palindrome while ignoring non-alphanumerics.

120. Write a program that sorts words in a string by their length.

1. Check if a number is a palindrome

```
def is_palindrome(n):  
    return str(n) == str(n)[::-1]
```

2. Find the largest of three numbers

```
def largest_of_three(a, b, c):  
    return max(a, b, c)
```

3. Recursive factorial function

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)
```

4. Remove duplicates from a list

```
def remove_duplicates(lst):  
    return list(set(lst))
```

5. Return the second largest number in a list

```
def second_largest(lst):  
    unique = list(set(lst))  
    if len(unique) < 2:  
        return None  
    unique.sort()  
    return unique[-2]
```

6. Reverse a string without built-in functions

```
def reverse_string(s):  
    reversed_str = ''  
    for char in s:  
        reversed_str = char + reversed_str  
    return reversed_str
```

7. Count the number of vowels in a string

```
def count_vowels(s):  
    vowels = 'aeiouAEIOU'  
    return sum(1 for char in s if char in vowels)
```

8. Check if two strings are anagrams

```
def is_anagram(str1, str2):  
    return sorted(str1) == sorted(str2)
```

9. Basic calculator using functions

```
def calculator(a, b, operator):  
    if operator == '+':  
        return a + b  
    elif operator == '-':  
        return a - b  
    elif operator == '*':  
        return a * b  
    elif operator == '/':  
        return a / b if b != 0 else "Division by zero"  
    else:  
        return "Invalid operator"
```

10. Merge two dictionaries

```
def merge_dicts(d1, d2):  
    return {**d1, **d2}
```

11. Count frequency of each word in a sentence

```
def word_frequency(sentence):  
    words = sentence.split()  
    freq = {}  
    for word in words:  
        freq[word] = freq.get(word, 0) + 1  
    return freq
```

12. Convert a binary string to decimal

```
def binary_to_decimal(binary_str):  
    return int(binary_str, 2)
```

13. Swap two numbers without a temporary variable

```
def swap(a, b):  
    a, b = b, a  
    return a, b
```

14. Check if a number is prime

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n ** 0.5) + 1):  
        if n % i == 0:  
            return False  
    return True
```

15. Sort a list of tuples by second element

```
def sort_by_second(tuples_list):  
    return sorted(tuples_list, key=lambda x: x[1])
```

16. Reverse words in a sentence

```
def reverse_words(sentence):  
    return ' '.join(sentence.split()[::-1])
```

17. Generate Fibonacci numbers up to n terms

```
def fibonacci(n):  
    fib = [0, 1]  
    for _ in range(2, n):
```

```
fib.append(fib[-1] + fib[-2])  
return fib[:n]
```

18. Lambda to square every number in a list

```
square = lambda x: [i**2 for i in x]
```

19. Use map() to convert list of strings to integers

```
def convert_to_int(str_list):  
    return list(map(int, str_list))
```

20. Filter even numbers using filter()

```
def filter_evens(lst):  
    return list(filter(lambda x: x % 2 == 0, lst))
```

21. Reduce a list to its sum using reduce()

```
from functools import reduce  
  
def sum_list(lst):  
    return reduce(lambda x, y: x + y, lst)
```

22. Find the common elements in two lists

```
def common_elements(list1, list2):  
    return list(set(list1) & set(list2))
```

23. Flatten a nested list

```
def flatten(nested_list):  
    flat_list = []  
    for sublist in nested_list:  
        if isinstance(sublist, list):  
            flat_list.extend(flatten(sublist))  
        else:  
            flat_list.append(sublist)  
    return flat_list
```

24. Convert a list of integers into a single integer

```
def list_to_int(lst):  
    return int(''.join(map(str, lst)))
```

25. Check if a number is an Armstrong number

```
def is_armstrong(num):  
    digits = [int(d) for d in str(num)]  
    return num == sum(d**len(digits) for d in digits)
```

26. Convert Celsius to Fahrenheit

```
def celsius_to_fahrenheit(c):  
    return (c * 9/5) + 32
```

27. Print multiplication table of a number

```
def multiplication_table(n):  
    for i in range(1, 11):  
        print(f"{n} x {i} = {n*i}")
```

28. Remove punctuation from a string

```
import string  
  
def remove_punctuation(s):  
    return ''.join(c for c in s if c not in string.punctuation)
```

29. Count the number of digits in an integer

```
def count_digits(n):  
    return len(str(abs(n)))
```

30. Calculate LCM of two numbers

```
def lcm(a, b):  
    def gcd(x, y):  
        while y:  
            x, y = y, x % y  
        return x  
    return abs(a * b) // gcd(a, b)
```

31. Convert a string to title case manually

```
def to_title_case(s):  
    return ' '.join(word[0].upper() + word[1:].lower() if word else '' for word in s.split())
```

32. Create a list of squares using list comprehension

```
squares = [x**2 for x in range(1, 11)]
```

33. Find all prime numbers between 1 and 100

```
def primes_upto_100():
```

```
primes = []
for num in range(2, 101):
    if all(num % i != 0 for i in range(2, int(num**0.5) + 1)):
        primes.append(num)
return primes
```

34. Return a dictionary of squares from 1 to n

```
def squares_dict(n):
    return {x: x**2 for x in range(1, n+1)}
```

35. Check whether a string is a valid identifier

```
def is_valid_identifier(s):
    return s.isidentifier()
```

36. Use bitwise AND, OR, XOR on two numbers

```
def bitwise_ops(a, b):
    return {
        "AND": a & b,
        "OR": a | b,
        "XOR": a ^ b
    }
```

37. Decorator to print execution time

```
import time

def timing_decorator(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"Execution time: {end - start:.6f} seconds")
        return result
    return wrapper
```

38. Extract numbers from a mixed string

```
def extract_numbers(s):
    return ''.join(filter(str.isdigit, s))
```

39. Function with variable length arguments to return sum

```
def variable_sum(*args):
    return sum(args)
```

40. Recursive GCD function

```
def gcd(a, b):
    if b == 0:
        return a
```

```
return gcd(b, a % b)
```

41. Check if a string contains all unique characters

```
def is_unique(s):  
    return len(set(s)) == len(s)
```

42. Use zip() to pair elements from two lists

```
def pair_lists(list1, list2):  
    return list(zip(list1, list2))
```

43. Return intersection of two sets

```
def intersection(set1, set2):  
    return set1 & set2
```

44. Simple login system with 3 tries

```
def login_system():  
    correct_username = "admin"  
    correct_password = "password"  
    for _ in range(3):
```



```
username = input("Username: ")
password = input("Password: ")
if username == correct_username and password == correct_password:
    print("Login successful!")
    return
else:
    print("Incorrect credentials. Try again.")
print("Access denied.")
```

45. Most frequent character in a string

```
from collections import Counter

def most_frequent_char(s):
    count = Counter(s)
    return count.most_common(1)[0][0]
```

46. Convert number to binary using recursion

```
def to_binary(n):
    if n == 0:
        return ''
    return to_binary(n // 2) + str(n % 2)
```

47. Create nested dictionary from two lists

```
def nested_dict(keys, values):
    return {k: {'value': v} for k, v in zip(keys, values)}
```

48. Rotate list left by k elements

```
def rotate_left(lst, k):  
    k = k % len(lst)  
    return lst[k:] + lst[:k]
```

49. Check if list is sorted in ascending order

```
def is_sorted(lst):  
    return lst == sorted(lst)
```

50. Generator for even numbers up to n

```
def even_numbers(n):  
    for i in range(2, n+1, 2):  
        yield i
```

51. Sum of diagonal elements in a matrix

```
def diagonal_sum(matrix):  
    return sum(matrix[i][i] for i in range(len(matrix)))
```

52. Matrix transpose using list comprehension

```
def transpose(matrix):  
    return [[row[i] for row in matrix] for i in range(len(matrix[0]))]
```

53. Remove all whitespace from string

```
def remove_whitespace(s):  
    return ''.join(s.split())
```

54. Chunk a list into sublists of size n

```
def chunk_list(lst, n):  
    return [lst[i:i+n] for i in range(0, len(lst), n)]
```

55. Count uppercase and lowercase letters

```
def count_case(s):  
    upper = sum(1 for c in s if c.isupper())  
    lower = sum(1 for c in s if c.islower())  
    return upper, lower
```

56. Find the longest word in a sentence

```
def longest_word(sentence):  
    words = sentence.split()  
    return max(words, key=len)
```

57. Implement stack using list

```
stack = []  
stack.append(10)  
stack.append(20)  
print(stack.pop())
```

58. Implement queue using collections.deque

```
from collections import deque  
  
queue = deque()  
queue.append(1)  
queue.append(2)  
print(queue.popleft())
```

59. Simulate rolling a dice

```
import random  
  
def roll_dice():  
    return random.randint(1, 6)
```

60. Count number of lines in a file

```
def count_lines(filename):  
    with open(filename, 'r') as f:  
        return len(f.readlines())
```

61. Read and reverse lines from file

```
def reverse_lines(filename):  
    with open(filename, 'r') as f:  
        return [line.strip()[::-1] for line in f]
```

62. Bubble sort

```
def bubble_sort(lst):  
    n = len(lst)  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if lst[j] > lst[j+1]:  
                lst[j], lst[j+1] = lst[j+1], lst[j]  
    return lst
```

63. Binary search

```
def binary_search(lst, target):  
    low, high = 0, len(lst)-1  
    while low <= high:  
        mid = (low + high) // 2  
        if lst[mid] == target:  
            return mid  
        elif lst[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1
```

64. Replace all occurrences of substring

```
def replace_all(s, old, new):  
    return s.replace(old, new)
```

65. Invert dictionary using comprehension

```
def invert_dict(d):  
    return {v: k for k, v in d.items()}
```

66. Extract domain name from email

```
def extract_domain(email):  
    return email.split('@')[-1]
```

67. Split string into chunks of length n

```
def split_chunks(s, n):  
    return [s[i:i+n] for i in range(0, len(s), n)]
```

68. Caesar cipher encryption/decryption

```
def caesar_cipher(text, shift):  
    result = ''  
    for char in text:  
        if char.isalpha():  
            base = ord('A') if char.isupper() else ord('a')  
            result += chr((ord(char) - base + shift) % 26 + base)  
        else:  
            result += char  
    return result
```

69. Use enumerate to iterate with index

```
def print_with_index(lst):  
    for i, val in enumerate(lst):  
        print(i, val)
```

70. Count function calls

```
def count_calls(func):
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@count_calls
def test():
    print("Called!")

# test(); test(); print(test.calls)
```

71. Find common prefix of strings

```
def common_prefix(strings):
    if not strings:
        return ""
    prefix = strings[0]
    for s in strings[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
    return prefix
```

72. Simulate a calculator

```
def calculator():
    expr = input("Enter expression: ")
    try:
        print("Result:", eval(expr))
```



```
except Exception as e:  
    print("Error:", e)
```

73. Remove all non-digit characters

```
def remove_non_digits(s):  
    return ''.join(filter(str.isdigit, s))
```

74. List all substrings of a string

```
def all_substrings(s):  
    return [s[i:j] for i in range(len(s)) for j in range(i+1, len(s)+1)]
```

75. Second most frequent word in file

```
from collections import Counter  
  
def second_most_frequent(filename):  
    with open(filename, 'r') as f:  
        words = f.read().split()  
        freq = Counter(words).most_common(2)  
        return freq[1][0] if len(freq) >= 2 else None
```

76. Custom exception

```
class CustomError(Exception):  
    pass  
  
def check_positive(n):  
    if n < 0:  
        raise CustomError("Negative number not allowed")
```

77. Validate IP address

```
def is_valid_ip(ip):  
    parts = ip.split('.')  
    if len(parts) != 4:  
        return False  
    for p in parts:  
        if not p.isdigit() or not (0 <= int(p) <= 255):  
            return False  
    return True
```

78. Smallest missing positive integer

```
def smallest_missing_positive(nums):  
    nums = set(nums)  
    i = 1  
    while i in nums:  
        i += 1  
    return i
```

79. Remove duplicates and sort

```
def unique_sorted(lst):  
    return sorted(set(lst))
```

80. Check for balanced parentheses

```
def is_balanced(s):  
    stack = []  
    mapping = {'(': ')', ')': '(', '[': ']', ']': '['}  
    for char in s:  
        if char in mapping.values():  
            stack.append(char)  
        elif char in mapping:  
            if not stack or stack.pop() != mapping[char]:  
                return False  
    return not stack
```

81. Group words that are anagrams of each other

```
from collections import defaultdict  
  
def group_anagrams(words):  
    anagrams = defaultdict(list)  
    for word in words:  
        key = ''.join(sorted(word))  
        anagrams[key].append(word)  
    return list(anagrams.values())
```

82. Convert Roman numerals to integers

```
def roman_to_int(s):  
    roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}  
    total = 0  
    prev = 0  
    for char in reversed(s):  
        value = roman[char]  
        if value < prev:  
            total -= value  
        else:  
            total += value  
        prev = value  
    return total
```

83. Check if a list is a palindrome

```
def is_palindrome_list(lst):  
    return lst == lst[::-1]
```

84. Create a histogram from a dictionary

```
def print_histogram(data):  
    for key, value in data.items():  
        print(f'{key}: {"*" * value}')
```

85. Insertion sort on a list

```
def insertion_sort(lst):  
    for i in range(1, len(lst)):  
        key = lst[i]  
        j = i - 1  
        while j >= 0 and key < lst[j]:  
            lst[j + 1] = lst[j]  
            j -= 1  
        lst[j + 1] = key  
    return lst
```

86. Reverse a dictionary (values become keys)

```
def reverse_dict(d):  
    return {v: k for k, v in d.items()}
```

87. Merge k sorted lists into a single sorted list

```
import heapq  
  
def merge_k_sorted(lists):  
    return list(heapq.merge(*lists))
```

88. Flatten a dictionary

```
def flatten_dict(d, parent_key='', sep='.'):  
    items = {}
```

```
for k, v in d.items():
    new_key = f'{parent_key}{sep}{k}' if parent_key else k
    if isinstance(v, dict):
        items.update(flatten_dict(v, new_key, sep=sep))
    else:
        items[new_key] = v
return items
```

89. Generate random password of length n

```
import random
import string

def generate_password(n):
    chars = string.ascii_letters + string.digits + string.punctuation
    return ''.join(random.choice(chars) for _ in range(n))
```

90. Factorial using lambda and reduce

```
from functools import reduce

factorial = lambda n: reduce(lambda x, y: x * y, range(1, n + 1), 1)
```

91. Simulate tossing a coin 100 times

```
import random

def toss_coin():
```

```
results = {'Heads': 0, 'Tails': 0}
for _ in range(100):
    result = 'Heads' if random.randint(0, 1) == 0 else 'Tails'
    results[result] += 1
return results
```

92. Longest increasing subsequence

```
def longest_increasing_subsequence(seq):
    n = len(seq)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if seq[i] > seq[j]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

93. Rotate matrix by 90 degrees clockwise

```
def rotate_matrix(matrix):
    return [list(reversed(col)) for col in zip(*matrix)]
```

94. Sum of even and odd digits

```
def sum_even_odd_digits(n):
    even = odd = 0
    for digit in str(n):
        d = int(digit)
```

```
if d % 2 == 0:
    even += d
else:
    odd += d
return even, odd
```

95. Convert number to English words

```
import inflect
p = inflect.engine()

def number_to_words(n):
    return p.number_to_words(n)
```

96. Power of number without **

```
def power(base, exp):
    result = 1
    for _ in range(abs(exp)):
        result *= base
    return result if exp >= 0 else 1 / result
```

97. Find the nth prime number

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0: return False
```



```
    return True

def nth_prime(n):
    count, num = 0, 1
    while count < n:
        num += 1
        if is_prime(num):
            count += 1
    return num
```

98. Mimic behavior of `range()`

```
def custom_range(start, stop=None, step=1):
    if stop is None:
        stop, start = start, 0
    while start < stop:
        yield start
        start += step
```

99. Quicksort

```
def quicksort(lst):
    if len(lst) <= 1:
        return lst
    pivot = lst[0]
    left = [x for x in lst[1:] if x < pivot]
    right = [x for x in lst[1:] if x >= pivot]
    return quicksort(left) + [pivot] + quicksort(right)
```

100. Average of numbers from user input

```
def average_input():  
    nums = list(map(float, input("Enter numbers separated by space: ").split()))  
    return sum(nums) / len(nums) if nums else 0
```

101. Replace spaces in a string with hyphens.

```
def replace_spaces_with_hyphens(s):  
    return s.replace(' ', '-')
```

102. Write a program that finds the square root of a number without using math.sqrt.

```
def sqrt_newton(n, tolerance=1e-10):  
    if n < 0:  
        return None # Square root not defined for negative numbers here  
    x = n  
    while True:  
        root = 0.5 * (x + n / x)  
        if abs(root - x) < tolerance:  
            return root  
    x = root
```

103. Implement a digital clock that shows time every second.

```
import time

def digital_clock():
    while True:
        print(time.strftime("%H:%M:%S"), end='\r')
        time.sleep(1)
```

104. Write a program to find all palindromic substrings of a string.

```
def find_palindromic_substrings(s):
    result = set()
    n = len(s)

    def expand(left, right):
        while left >= 0 and right < n and s[left] == s[right]:
            result.add(s[left:right+1])
            left -= 1
            right += 1

    for i in range(n):
        expand(i, i)      # Odd length palindromes
        expand(i, i+1)    # Even length palindromes

    return result
```

105. Find the number of trailing zeroes in factorial of a number.

```
def trailing_zeroes_factorial(n):
    count = 0
    i = 5
    while n >= i:
        count += n // i
```

```
i *= 5  
return count
```

106. Convert a list of strings to a CSV string.

```
def list_to_csv(lst):  
    return ','.join(lst)
```

107. Implement a function to compress a string using counts of repeated characters.

```
def compress_string(s):  
    if not s:  
        return ''  
    compressed = []  
    count = 1  
    for i in range(1, len(s)):  
        if s[i] == s[i-1]:  
            count += 1  
        else:  
            compressed.append(s[i-1] + str(count))  
            count = 1  
    compressed.append(s[-1] + str(count))  
    return ''.join(compressed)
```

108. Write a function to evaluate a postfix expression.

```
def evaluate_postfix(expression):  
    stack = []
```

```
for token in expression.split():
    if token.isdigit():
        stack.append(int(token))
    else:
        b = stack.pop()
        a = stack.pop()
        if token == '+': stack.append(a + b)
        elif token == '-': stack.append(a - b)
        elif token == '*': stack.append(a * b)
        elif token == '/': stack.append(a / b)
return stack.pop()
```

109. Extract all hashtags from a string.

```
import re

def extract_hashtags(s):
    return re.findall(r'#\w+', s)
```

110. Count frequency of characters using collections.Counter.

```
from collections import Counter

def char_frequency(s):
    return Counter(s)
```

111. Reverse a string using a stack.

```
def reverse_string_stack(s):  
    stack = list(s)  
    reversed_str = ''  
    while stack:  
        reversed_str += stack.pop()  
    return reversed_str
```

112. Implement a cache using dictionary with a size limit.

```
from collections import OrderedDict  
  
class Cache:  
    def __init__(self, size_limit):  
        self.size_limit = size_limit  
        self.cache = OrderedDict()  
  
    def get(self, key):  
        if key in self.cache:  
            self.cache.move_to_end(key)  
            return self.cache[key]  
        return None  
  
    def put(self, key, value):  
        if key in self.cache:  
            self.cache.move_to_end(key)  
        self.cache[key] = value  
        if len(self.cache) > self.size_limit:  
            self.cache.popitem(last=False)
```

113. Write a function to multiply all elements in a nested list.

```
def multiply_nested(lst):
```

```
result = 1
for item in lst:
    if isinstance(item, list):
        result *= multiply_nested(item)
    else:
        result *= item
return result
```

114. Count how many times a function is called using a decorator.

```
def call_counter(func):
    def wrapper(*args, **kwargs):
        wrapper.count += 1
        return func(*args, **kwargs)
    wrapper.count = 0
    return wrapper

@call_counter
def example_function():
    pass
```

example_function() called multiple times, then example_function.count has number of calls

115. Find duplicate files in a directory by content.

```
import os
import hashlib

def find_duplicate_files(directory):
    hashes = {}
    duplicates = []
```

```
for root, _, files in os.walk(directory):
    for file in files:
        path = os.path.join(root, file)
        with open(path, 'rb') as f:
            file_hash = hashlib.md5(f.read()).hexdigest()
            if file_hash in hashes:
                duplicates.append((hashes[file_hash], path))
            else:
                hashes[file_hash] = path
return duplicates
```

116. Write a program that mimics the UNIX wc command.

```
def wc(filename):
    lines = words = chars = 0
    with open(filename, 'r') as f:
        for line in f:
            lines += 1
            words += len(line.split())
            chars += len(line)
    return lines, words, chars
```

117. Create a function that finds the median of a list.

```
def median(lst):
    sorted_lst = sorted(lst)
    n = len(sorted_lst)
    mid = n // 2
    if n % 2 == 0:
        return (sorted_lst[mid - 1] + sorted_lst[mid]) / 2
    else:
        return sorted_lst[mid]
```

118. Simulate a progress bar using print() and time delay.

```
import time
import sys

def progress_bar(total):
    for i in range(total + 1):
        percent = (i / total) * 100
        bar = '#' * i + '-' * (total - i)
        print(f'\r[bar] {percent:.2f}%', end='')
        sys.stdout.flush()
        time.sleep(0.1)
    print()
```

119. Detect if a string is a valid palindrome while ignoring non-alphanumerics.

```
import re

def is_valid_palindrome(s):
    s = re.sub(r'[^\w-0-9]', '', s).lower()
    return s == s[::-1]
```

120. Write a program that sorts words in a string by their length.

```
def sort_words_by_length(s):
    words = s.split()
    words.sort(key=len)
```

```
return ' '.join(words)
```



